

MAT61-3

Math for ML

Kernels, Support Vector Machines, Regularization, Attention &
Reinforcement Learning

Synthesizing linear algebra, probability, and optimization into the mathematical toolkit
of modern machine learning.

Albert School — 30 hours

Preface

You arrive at this course already holding every tool it needs. From **Linear Algebra IV** (MAT41-1) you carry the singular value decomposition, eigendecomposition of symmetric matrices, and principal component analysis — the machinery of projecting data onto the directions that matter. From **Advanced Estimation** (MAT41-2) you carry bias, variance, mean squared error, and the tradeoff between them. From **NN & Deep Learning Math** (MAT41-3) you carry the softmax map, cross-entropy, residual connections, and layer normalization. From **Optimization** (MAT61-1) you carry convexity, the KKT conditions, gradient descent, and the proximal/soft-thresholding operator. And from **Probability & Statistics II** (MAT31-2) you carry the Markov property and the Bellman equation, written there as a one-step accounting identity on a single small example.

This capstone does not introduce a new branch of mathematics. Instead it **assembles** what you have into the foundations of classical machine learning and of the attention mechanism that powers Transformers. The throughline is a single idea, met again and again in different costumes: **a learning algorithm is an optimization problem, and the geometry of that problem — its convexity, its constraints, its inner products — determines what the algorithm can and cannot learn.**

We proceed in four movements. First, the **theory of generalization**: bias-variance, regularization, and cross-validation — how we measure and control the gap between fitting the data we have and predicting the data we do not. Second, **kernel methods**: how a single trick — replacing inner products by kernel evaluations — turns linear algorithms (SVMs, PCA) into nonlinear ones. Third, **attention**, revealed as a kernel operation in a learned embedding space. Fourth, **reinforcement learning**, where the Bellman equation you already know becomes an algorithm. Throughout, every new idea is met first as a concrete example and only then stated abstractly.

Contents

Preface	2
1 The bias-variance framework	4
1.1 A concrete failure of memorization	4
1.2 Decomposing the expected prediction error	4
1.3 Underfitting, overfitting, and the two error curves	5
2 Regularization	5
2.1 Ridge regression: the L_2 penalty	6
2.1.1 Effect on the eigenvalues — shrinkage in the SVD basis	6
2.2 Lasso: the L_1 penalty and sparsity	7
2.2.1 Why there is no closed form — and what replaces it	7
2.3 L_1 vs L_2 : the geometry of the constraint	7
3 Cross-validation	8
3.1 k -fold cross-validation	8
3.2 Leave-one-out, and the model-selection procedure	8
4 Kernel functions	9
4.1 A concrete impossibility, and its cure	9
4.2 The kernel trick	9
4.3 Which functions are kernels? The Gram matrix and Mercer’s condition	10
4.4 The standard kernels and their hyperparameters	11
5 Support Vector Machines	11

5.1	The maximum-margin idea	11
5.2	Hard-margin SVM	12
5.3	The dual, via KKT — and why it unlocks kernels	12
5.4	Soft margin: tolerating overlap	13
6	Kernel PCA	13
6.1	From covariance to the Gram matrix	13
6.2	Centering in feature space	14
6.3	The algorithm	14
6.4	Computational tradeoff: m^3 vs. d^3	14
7	Attention as a kernel operation	14
7.1	Scaled dot-product attention	14
7.2	The kernel reading	15
7.3	Softmax as a Boltzmann distribution over keys	15
7.4	Multi-head attention	16
7.5	The Transformer block as a mathematical object	16
8	Reinforcement learning	16
8.1	The Markov decision process	17
8.2	The Bellman equations, now as fixed points	17
8.3	Solving a known MDP: value and policy iteration	17
8.4	Learning without a model: bootstrapping and Q-learning	18
8.5	Exploration and off-policy learning	19
9	Exercises	20

1 The bias-variance framework

Before studying any specific learning algorithm, we need a language for what it means to **learn well**. A model that reproduces its training data perfectly may predict new data terribly; a model too crude to fit the training data will fail everywhere. The bias-variance framework makes this tension precise.

1.1 A concrete failure of memorization

Suppose we collect $n = 10$ points (x_i, y_i) generated by

$$y = \sin(2\pi x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

and we try to recover the curve with a polynomial. A degree-1 line is too rigid: it cannot bend with the sine, so it is wrong everywhere in the same systematic way — this is **underfitting**. A degree-9 polynomial passes through all ten points exactly, but between them it oscillates wildly, chasing the noise ε rather than the signal $\sin(2\pi x)$ — this is **overfitting**. Somewhere in between, a degree-3 or degree-4 polynomial captures the shape without chasing the noise.

The key observation: the degree-9 fit has **zero training error** and yet large **test error**. Training error alone cannot tell us whether we have learned. We need to decompose the error into pieces that explain why.

1.2 Decomposing the expected prediction error

Fix an input $\mathbf{x}$. The true label is $y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = \sigma^2$. Our model, trained on a random dataset D , produces a prediction $\hat{f}(\mathbf{x})$ — random because D is random. We measure error in expectation over both the noise and the draw of D .

Theorem 1 (Bias-variance decomposition) For squared-error loss, the expected prediction error at $\mathbf{x}$ splits into three non-negative terms:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{\sigma^2}_{\text{irreducible}} + \underbrace{(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2]}_{\text{variance}}.$$

Proof. Write $\hat{f} = \hat{f}(\mathbf{x})$ and $\bar{f} = \mathbb{E}[\hat{f}]$. Since $y = f + \varepsilon$ with ε independent of D and mean zero,

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f + \varepsilon - \hat{f})^2] = \mathbb{E}[\varepsilon^2] + \mathbb{E}[(f - \hat{f})^2],$$

the cross term $2\mathbb{E}[\varepsilon(f - \hat{f})] = 0$ vanishing by independence and $\mathbb{E}[\varepsilon] = 0$. The first piece is σ^2 . For the second, insert and subtract $\bar{f}$:

$$\mathbb{E}[(f - \hat{f})^2] = \mathbb{E}[(f - \bar{f}) + (\bar{f} - \hat{f})^2] = (f - \bar{f})^2 + \mathbb{E}[(\bar{f} - \hat{f})^2],$$

the cross term vanishing because $f - \bar{f}$ is constant and $\mathbb{E}[\bar{f} - \hat{f}] = 0$. These are bias^2 and variance. $\square$

Read the three terms as a budget you cannot escape:

- **Irreducible error** σ^2 is the noise floor. No model, however good, can predict ε . This term is why a test error of exactly zero is impossible on noisy data, and why chasing it produces overfitting.
- **Bias** measures how far the **average** model $\mathbb{E}[\hat{f}]$ sits from the truth. High bias = the model class is too rigid (the line vs. the sine).

- **Variance** measures how much the model **wiggles** as the training set changes. High variance = the model is so flexible it reshapes itself around each new dataset (the degree-9 polynomial).

This connects directly to the bias/MSE/variance vocabulary of **Advanced Estimation** (MAT41-2): there $\hat{f}$ was a parameter estimator; here it is a whole prediction function, but the algebra — $\text{MSE} = \text{bias}^2 + \text{variance}$ — is identical.

1.3 Underfitting, overfitting, and the two error curves

As we increase model complexity (polynomial degree, network width, number of features), bias falls — the model can represent richer truths — but variance rises — the model has more freedom to fit noise. **Training error** falls monotonically toward zero, while **test error** traces a U: it falls while reducing bias helps, then rises once variance dominates. The bottom of the U is the sweet spot.

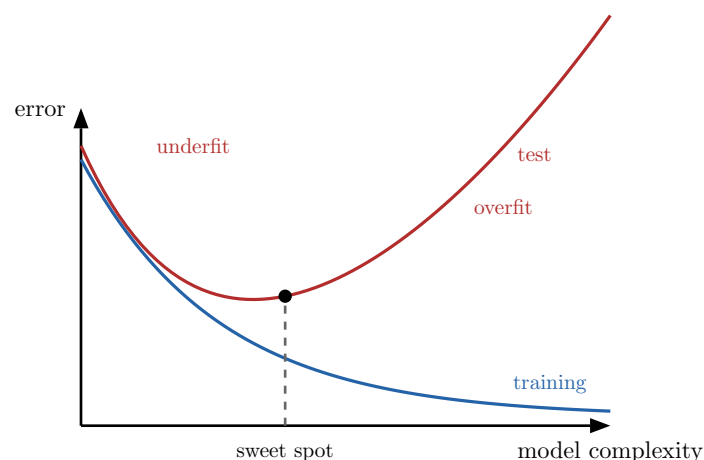


Figure 1: Training error falls monotonically; test error is U-shaped. The minimum of the test curve marks the best bias-variance compromise. The gap between the two curves is a diagnostic of variance.

Diagnosis rule of thumb. High training error → high bias (underfitting): use a richer model or more features. Low training error but high test error, with a large gap between them → high variance (overfitting): use more data, fewer features, or **regularization** — the subject of the next chapter.

Common pitfall The decomposition is an **expectation** over hypothetical re-draws of the training set; you never observe bias and variance separately from one dataset. You estimate the test error empirically (Chapter 3) and read bias vs. variance from the **shape** of the train/test curves, not from a single number. Also: the irreducible σ^2 does not vanish with more data — only the variance term does.

2 Regularization

Overfitting is excess variance: the model uses its freedom to fit noise. Regularization fights back by **penalizing complexity**, trading a little bias for a large reduction in variance. We add to the data-fitting loss a penalty on the size of the parameters $\mathbf{w}$:

$$\operatorname{argmin}_{\mathbf{w}} \underbrace{\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2}_{\text{fit}} + \lambda \cdot \underbrace{R(\mathbf{w})}_{\text{penalty}}.$$

The hyperparameter $\lambda \geq 0$ sets the exchange rate. The two classic choices $R(\mathbf{w}) = \|\mathbf{w}\|_2^2$ and $R(\mathbf{w}) = \|\mathbf{w}\|_1$ behave very differently — one shrinks, the other selects.

2.1 Ridge regression: the L_2 penalty

Definition 2 (Ridge regression) Ridge regression solves

$$\hat{\mathbf{w}}_{\text{ridge}} = \operatorname{argmin}_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

Because both terms are convex and the penalty is **strictly** convex, the objective is strictly convex, so by the uniqueness result of MAT61-1 the minimizer is unique — even when $\mathbf{X}^T \mathbf{X}$ is singular. Setting the gradient to zero,

$$2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda\mathbf{w} = \mathbf{0} \Rightarrow (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})\mathbf{w} = \mathbf{X}^T \mathbf{y},$$

giving the **closed form**

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}.$$

The added $\lambda \mathbf{I}$ is what makes the inverse always exist: it lifts every eigenvalue of $\mathbf{X}^T \mathbf{X}$ by $\lambda > 0$, curing the ill-conditioning that, in ordinary least squares, blows the variance up (recall from MAT41-1 that a tiny singular value σ_r makes the condition number σ_1/σ_r explode).

2.1.1 Effect on the eigenvalues — shrinkage in the SVD basis

To **see** what ridge does, write the SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ from MAT41-1. The ridge prediction on the training inputs is

$$\mathbf{X}\hat{\mathbf{w}}_{\text{ridge}} = \sum_{i=1}^r \mathbf{u}_i \left(\frac{\sigma_i^2}{\sigma_i^2 + \lambda} \right) \mathbf{u}_i^T \mathbf{y}.$$

Proof (sketch). Substitute the SVD into the closed form. Then $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} = \mathbf{V}(\mathbf{\Sigma}^T \mathbf{\Sigma} + \lambda \mathbf{I})\mathbf{V}^T$, and after cancelling $\mathbf{V}$ factors each singular direction $\mathbf{u}_i$ receives the coefficient $\sigma_i/(\sigma_i^2 + \lambda)$ when multiplied by $\mathbf{X}$, i.e. a filter factor $\sigma_i^2/(\sigma_i^2 + \lambda)$ on the component $\mathbf{u}_i^T \mathbf{y}$. $\square$

The factor $f_i = \sigma_i^2/(\sigma_i^2 + \lambda) \in (0, 1)$ is a **shrinkage filter**:

- For **large** σ_i (high-variance directions, well-determined by the data), $f_i \approx 1$: ridge leaves these almost untouched.
- For **small** σ_i (directions the data barely constrains, where OLS would produce huge, noisy coefficients), $f_i \approx \sigma_i^2/\lambda \approx 0$: ridge crushes them.

This is the **shrinkage interpretation**: ridge does not treat all parameters equally; it selectively shrinks the directions where the data is uninformative, which is exactly where overfitting lives.

Worked example — why shrinkage reduces variance. Two nearly-collinear features make $\mathbf{X}^T \mathbf{X}$ nearly singular, so OLS can place enormous positive weight on one and enormous negative weight on the other (they cancel). A new data point breaks the cancellation and the prediction swings wildly — high variance. Ridge forbids large $\|\mathbf{w}\|_2$, so it splits the weight modestly between the two features. The price is a little bias; the reward is a large drop in variance.

2.2 Lasso: the L_1 penalty and sparsity

Definition 3 (Lasso) The lasso solves

$$\hat{\mathbf{w}}_{\text{lasso}} = \operatorname{argmin}_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 = \sum_j |w_j|.$$

The lasso looks like ridge with the 2-norm replaced by the 1-norm, but the consequence is qualitatively different: **the lasso sets coefficients exactly to zero**, performing automatic **feature selection**. Ridge shrinks coefficients toward zero but, generically, never to zero.

2.2.1 Why there is no closed form — and what replaces it

The penalty $\|\mathbf{w}\|_1$ is convex but **not differentiable** at any $w_j = 0$, so we cannot just set a gradient to zero. There is no closed-form solution. Instead we use the **proximal** machinery of MAT61-1. In the special orthonormal case $\mathbf{X}^T \mathbf{X} = \mathbf{I}$, the lasso solution is the ordinary least-squares solution passed through the **soft-thresholding** operator — exactly the proximal operator of the L_1 norm you derived there:

$$(\hat{\mathbf{w}}_{\text{lasso}})_j = \operatorname{soft}_{\lambda/2}(\hat{w}_j^{\text{ols}}) = \operatorname{sign}(\hat{w}_j^{\text{ols}}) \left(|\hat{w}_j^{\text{ols}}| - \frac{\lambda}{2} \right)_+,$$

where $(t)_+ = \max(t, 0)$. Soft-thresholding moves every coefficient toward zero by a fixed amount $\lambda/2$ and **clips to zero** anything smaller than the threshold — the mechanism that creates sparsity. In the general case one runs ISTA/FISTA (MAT61-1) to the same effect.

2.3 L_1 vs L_2 : the geometry of the constraint

Both penalties have a constrained reading: minimize $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ subject to $\|\mathbf{w}\| \leq t$. The least-squares term has **elliptical** contours centered on the OLS solution $\hat{\mathbf{w}}^{\text{ols}}$. The solution is where the smallest such ellipse first touches the constraint region. The shape of that region decides everything.

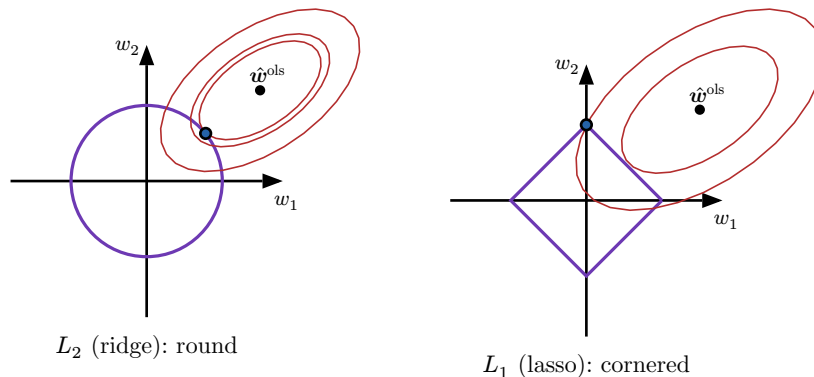


Figure 2: Elliptical loss contours meet the constraint region at the blue point. The round L_2 ball is generically touched at a point with **both** coordinates nonzero (shrinkage). The L_1 diamond has sharp vertices **on the axes**, so contours tend to meet it at a corner where a coordinate is exactly zero (sparsity / feature selection).

The diamond's corners lie on the axes, and corners stick out — an expanding ellipse is far more likely to strike a protruding vertex than a flat edge. At a vertex, one coordinate is exactly 0. The smooth circle has no such corners, so the tangency lands generically off the axes: both coordinates stay nonzero, just smaller.

When to use which.

- **Ridge (L_2):** many features that are all somewhat relevant; correlated predictors you want to keep but tame; you want a stable closed-form solution. Shrinks, never selects.
- **Lasso (L_1):** you believe only a few features truly matter and want an interpretable, **sparse** model that selects them automatically. No closed form; solved by proximal methods.
- **In between:** the elastic net combines both penalties (mentioned for completeness).

Common pitfall Always **standardize** features before regularizing. Both penalties act on the raw coefficient magnitudes, so a feature measured in millimetres is penalized $1000 \times$ more lightly than the same feature in metres. Without standardization, λ has no consistent meaning across features.

3 Cross-validation

Regularization introduces a knob λ ; kernels will introduce γ and a degree; SVMs will introduce C . How do we **choose** these hyperparameters? Not by training error — that always prefers the most complex model. Not by the test set — touching it for tuning contaminates our final estimate. We need an honest estimate of test error computed from the training data alone. That is cross-validation.

3.1 k -fold cross-validation

Definition 4 (k -fold cross-validation) Partition the training data into k equal **folds**. For each fold $i = 1, \dots, k$: train on the other $k - 1$ folds and evaluate on fold i , obtaining an error e_i . The cross-validation estimate is the average

$$\text{CV}_k = \frac{1}{k} \sum_{i=1}^k e_i.$$

Every data point is used for validation exactly once and for training $k - 1$ times, so we extract an honest held-out estimate from all the data — no point is wasted, yet no point is ever validated by a model that trained on it.

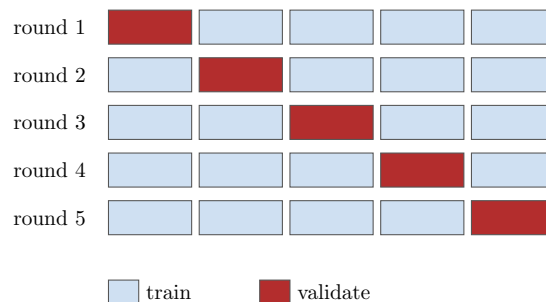


Figure 3: 5-fold cross-validation. Each row is one training round; the red block is the held-out validation fold for that round. Averaging the five validation errors gives CV_5 .

3.2 Leave-one-out, and the model-selection procedure

Leave-one-out (LOO) is the extreme $k = n$: each fold is a single point. It is nearly unbiased (each model trains on $n - 1$ points, almost the full dataset) but has higher variance and costs n

fits — expensive unless a shortcut exists (for ridge, a closed-form LOO formula avoids refitting). Common practice uses $k = 5$ or $k = 10$ as a compromise between bias, variance, and compute.

Model-selection procedure.

1. Split off a **test set** and lock it away.
2. For each candidate hyperparameter value (e.g. each λ on a grid), compute CV_k on the training set.
3. Pick the hyperparameter minimizing CV_k .
4. Refit the chosen model on the **full** training set.
5. Report performance **once** on the locked test set.

Common pitfall The cardinal sin is **data leakage**. Any choice informed by the validation fold must be made **inside** the cross-validation loop — including feature scaling, feature selection, and imputation. Standardizing on the whole dataset before splitting lets information from the validation fold leak into training and produces optimistically biased estimates. Likewise, never tune on the final test set: once you select using it, it is no longer an unbiased estimate of generalization.

4 Kernel functions

Linear models are easy to fit but can only draw straight boundaries. The kernel trick is the single most elegant idea for making them nonlinear **without paying the price of working in a high-dimensional space**. It is the engine behind kernel SVMs, kernel PCA, and — as we will see — a useful lens on attention.

4.1 A concrete impossibility, and its cure

Consider four points in the plane: $(0,0)$ and $(1,1)$ labelled -1 ; $(1,0)$ and $(0,1)$ labelled $+1$. This is the XOR pattern, and **no straight line separates the two classes**. A linear classifier is helpless.

Now map each point through the feature map

$$\varphi(x_1, x_2) = (x_1, x_2, x_1x_2) \in \mathbb{R}^3.$$

The new third coordinate x_1x_2 equals 0 for the positive class and 0 or 1... let us instead use $\varphi(x_1, x_2) = (x_1, x_2, (x_1 - x_2)^2)$. Then the positive points get third coordinate 1 and the negative points get 0: a plane at height $\frac{1}{2}$ separates them cleanly. **Lifting the data into a richer feature space turned an impossible linear problem into an easy one.** The catch: useful feature maps can be enormous (a degree- d polynomial map of $\mathbb{R}^p$ has $O(p^d)$ coordinates), and computing $\varphi(\mathbf{x})$ explicitly becomes prohibitive.

4.2 The kernel trick

Here is the rescue. Many linear algorithms — least squares, PCA, the SVM dual — never need the feature vectors $\varphi(\mathbf{x})$ themselves; **they only ever use inner products** $\langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle$. If we can compute that inner product **directly**, as a function of $\mathbf{x}$ and $\mathbf{z}$, we never build φ at all.

Definition 5 (Kernel) A **kernel** is a function $k(\mathbf{x}, \mathbf{z})$ that equals an inner product in some feature space: $k(\mathbf{x}, \mathbf{z}) = \langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle$ for some map φ .

Worked example — the quadratic kernel computes a 3-D inner product in 2-D. Take $\varphi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$. Then

$$\langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = (x_1 z_1 + x_2 z_2)^2 = (\mathbf{x}^T \mathbf{z})^2.$$

So the kernel $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^2$ secretly evaluates a three-dimensional inner product using only a two-dimensional dot product and a squaring. This is the **kernel trick**: replace every $\langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle$ by $k(\mathbf{x}, \mathbf{z})$ and skip the feature map.

4.3 Which functions are kernels? The Gram matrix and Mercer’s condition

Not every symmetric k is an inner product in disguise. The gatekeeping condition is positive semi-definiteness of the **Gram matrix**.

Definition 6 (Gram matrix) Given data $\mathbf{x}_1, \dots, \mathbf{x}_m$ and a symmetric function k , the **Gram matrix** is $\mathbf{K} \in \mathbb{R}^{m \times m}$ with entries $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

Definition 7 (Positive semi-definite (PSD) kernel) A symmetric k is **positive semi-definite** if for **every** finite set of points the Gram matrix $\mathbf{K}$ is PSD, i.e. $\mathbf{c}^T \mathbf{K} \mathbf{c} \geq 0$ for all $\mathbf{c} \in \mathbb{R}^m$.

Theorem 8 (Mercer’s condition) A symmetric function k is a valid kernel — it can be written as $\langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle$ for some feature map φ into a (possibly infinite-dimensional) inner-product space — **if and only if** it is positive semi-definite.

The intuition for one direction is immediate: if $k(\mathbf{x}, \mathbf{z}) = \langle \varphi(\mathbf{x}), \varphi(\mathbf{z}) \rangle$, then for any coefficients $\mathbf{c}$,

$$\mathbf{c}^T \mathbf{K} \mathbf{c} = \sum_{i,j} c_i c_j \langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j) \rangle = \left\| \sum_i c_i \varphi(\mathbf{x}_i) \right\|^2 \geq 0,$$

so a genuine inner-product kernel is automatically PSD. Mercer’s theorem is the deep converse: PSD is also **sufficient** — every PSD kernel arises from some feature map. This is why we can design or verify kernels by **reasoning about the Gram matrix**, never needing to exhibit φ .

Worked example — verifying validity via the Gram matrix. Is $k(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^T \mathbf{z})^2$ a valid kernel? Expand: $(1 + \mathbf{x}^T \mathbf{z})^2 = 1 + 2\mathbf{x}^T \mathbf{z} + (\mathbf{x}^T \mathbf{z})^2$. Each summand is a valid kernel (a constant, a positive multiple of the linear kernel, and the quadratic kernel above), and sums of PSD kernels are PSD, so k is valid. By contrast $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z} - 1$ is **not** PSD (the one-point Gram “matrix” at $\mathbf{x} = \mathbf{0}$ is $-1 < 0$), so it is not a kernel.

Kernel construction rules. If k_1, k_2 are kernels, then so are $k_1 + k_2$, αk_1 for $\alpha \geq 0$, $k_1 \cdot k_2$, and $f(\mathbf{x})k_1(\mathbf{x}, \mathbf{z})f(\mathbf{z})$ for any function f . These let you build and verify complex kernels without ever touching a feature map.

4.4 The standard kernels and their hyperparameters

Definition 9 (Standard kernels)

- **Linear:** $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$. Feature map is the identity; recovers ordinary linear models.
- **Polynomial:** $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$. Feature map contains all monomials up to degree d ; $c \geq 0$ trades off low- vs. high-order terms.
- **RBF (Gaussian):** $k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$, $\gamma > 0$. Its feature space is **infinite-dimensional**.

The RBF kernel is worth a second look: it measures **similarity by distance** — $k = 1$ when $\mathbf{x} = \mathbf{z}$ and decays smoothly to 0 as they separate. It is a valid kernel (it can be shown PSD via the series expansion of the exponential), yet by Mercer it corresponds to an **infinite-dimensional** φ — which we could never compute explicitly. The kernel trick is not a convenience here; it is the only way.

Hyperparameter effects. The kernel's hyperparameters control the bias-variance tradeoff of Chapter 1:

- RBF γ : large $\gamma \rightarrow$ narrow bumps, each point influences only its immediate neighbours $\rightarrow$ very flexible, **high variance** (overfitting). Small $\gamma \rightarrow$ wide, smooth influence $\rightarrow$ **high bias** (underfitting).
- Polynomial degree d : higher $d \rightarrow$ more wiggly boundaries $\rightarrow$ higher variance.

Choose them by cross-validation (Chapter 3).

5 Support Vector Machines

The SVM is the flagship kernel method: a linear classifier whose objective is engineered so that (a) it generalizes well by maximizing a **margin**, and (b) it depends on the data only through inner products, so the kernel trick makes it nonlinear for free.

5.1 The maximum-margin idea

Among the infinitely many hyperplanes that separate two linearly separable classes, which is best? The SVM's answer: the one with the widest **margin** — the largest empty corridor around the boundary. A wide margin is robust; small perturbations of the data do not flip a point across the boundary.

A hyperplane is $\mathbf{w}^T \mathbf{x} + b = 0$. If we scale $(\mathbf{w}, b)$ so that the closest points satisfy $|\mathbf{w}^T \mathbf{x} + b| = 1$, the distance from such a point to the hyperplane is $1/\|\mathbf{w}\|$, so the full margin width is $2/\|\mathbf{w}\|$. Maximizing the margin means **minimizing** $\|\mathbf{w}\|$.

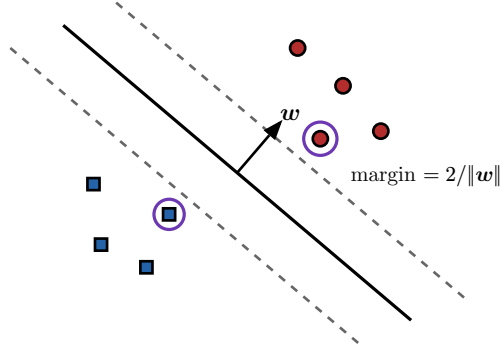


Figure 4: Maximum-margin classifier. The solid line is the decision boundary; the dashed lines are the margins. The circled points touching the margins are the **support vectors** — the only points that determine the solution. The weight vector $\mathbf{w}$ is normal to the boundary.

5.2 Hard-margin SVM

Definition 10 (Hard-margin SVM (primal)) For linearly separable data $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$,

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i.$$

The constraint $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ says every point lies on the correct side, **beyond** the margin. The objective is convex and the constraints are linear, so this is a convex quadratic program — exactly the well-behaved landscape MAT61-1 taught us to trust.

5.3 The dual, via KKT — and why it unlocks kernels

Form the Lagrangian with multipliers $\alpha_i \geq 0$ (the KKT framework of MAT61-1):

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1).$$

Stationarity $\partial \mathcal{L} / \partial \mathbf{w} = \mathbf{0}$ and $\partial \mathcal{L} / \partial b = 0$ give the two key relations

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i, \quad \sum_i \alpha_i y_i = 0.$$

Substituting back eliminates $\mathbf{w}$ and b and yields the **dual**:

Theorem 11 (Hard-margin SVM (dual))

$$\max_{\alpha \geq 0} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0.$$

Two features of the dual are decisive.

First, the data appears only through inner products $\mathbf{x}_i^T \mathbf{x}_j$. Replace each by $k(\mathbf{x}_i, \mathbf{x}_j)$ and the **same** solver produces a nonlinear classifier in the kernel's feature space — this is **kernel SVM**. The decision function is likewise kernelized:

$$\hat{y}(\mathbf{x}) = \text{sign} \left(\sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \right).$$

We never compute φ ; we only evaluate k .

Second, support vectors. The KKT **complementary slackness** condition $\alpha_i(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0$ forces $\alpha_i = 0$ for every point strictly beyond the margin. Only points **on** the margin have $\alpha_i > 0$. These are the **support vectors**; $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ depends on them alone. The solution is **sparse in the data points** — discard all the rest and the classifier is unchanged.

5.4 Soft margin: tolerating overlap

Real data is rarely perfectly separable, and even when it is, a hard margin can overfit a few outliers. The **soft-margin** SVM introduces **slack variables** $\xi_i \geq 0$ that let points violate the margin, at a price:

Definition 12 (Soft-margin SVM (primal))

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Each ξ_i measures how far point i intrudes into (or past) the margin. The **C parameter** sets the penalty for these intrusions and is the SVM’s bias-variance dial:

- **Large C** : violations are expensive, so the model strains to classify every training point correctly — narrow margin, **low bias, high variance** (overfitting).
- **Small C** : violations are cheap, so the model accepts some errors for a wider, smoother margin — **high bias, low variance** (underfitting).

The dual is almost identical to the hard-margin dual; the only change is that the multipliers are now **box-constrained**, $0 \leq \alpha_i \leq C$. Inner products still appear alone, so kernelization is unaffected. Choose C (and any kernel hyperparameters) by cross-validation.

Common pitfall C and the RBF γ interact strongly and must be tuned **together** (a 2-D grid search), not one at a time. A common mistake is to fix one at a default and sweep the other, missing the joint optimum. Also: do not confuse a **support vector** (any point with $\alpha_i > 0$) with a **misclassified** point; with soft margins, points on the margin, inside the margin, and across the boundary can all be support vectors.

6 Kernel PCA

Linear PCA (MAT41-1) finds the orthogonal directions of greatest variance by eigendecomposing the covariance matrix. But variance along **straight** directions cannot capture data that lies on a **curved** manifold (points on a spiral, say). Kernel PCA applies the kernel trick to PCA: it performs linear PCA in the feature space, where the curve may unfold into something flat.

6.1 From covariance to the Gram matrix

In linear PCA we eigendecompose the $d \times d$ covariance matrix. In feature space the “covariance” is $\frac{1}{m} \sum_i \varphi(\mathbf{x}_i) \varphi(\mathbf{x}_i)^T$ — possibly **infinite-dimensional**, so we cannot form it. The trick (exactly as in MAT41-1’s SVD/eigendecomposition link, where the eigenvectors of $\mathbf{X}\mathbf{X}^T$ and $\mathbf{X}^T\mathbf{X}$ are related) is to work instead with the $m \times m$ **Gram matrix** $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, whose eigenvectors encode the same principal components.

6.2 Centering in feature space

PCA requires **centered** data, but we cannot subtract the feature-space mean $\bar{\varphi} = \frac{1}{m} \sum_i \varphi(\mathbf{x}_i)$ explicitly. We center the Gram matrix instead:

Definition 13 (Centered Gram matrix) With $\mathbf{1}_m$ the $m \times m$ matrix of all $1/m$, the centered Gram matrix is

$$\tilde{K} = K - \mathbf{1}_m K - K \mathbf{1}_m + \mathbf{1}_m K \mathbf{1}_m.$$

This algebra is exactly the statement “subtract the mean from each $\varphi(\mathbf{x}_i)$ ”, carried out purely in terms of kernel evaluations.

6.3 The algorithm

Kernel PCA.

1. Compute the Gram matrix K , $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.
2. Center it: $\tilde{K}$ as above.
3. Eigendecompose: $\tilde{K} \mathbf{a}_l = m \lambda_l \mathbf{a}_l$, taking the top eigenvectors $\mathbf{a}_l$.
4. Normalize so the corresponding feature-space component is unit length: $\mathbf{a}_l^T \mathbf{a}_l = 1/(m \lambda_l)$.
5. Project a point $\mathbf{x}$ onto component l using only the kernel:

$$z_{l(\mathbf{x})} = \sum_{i=1}^m (a_l)_i k(\mathbf{x}_i, \mathbf{x}) \quad (\text{after centering}).$$

Every step uses kernel values only — never the (possibly infinite-dimensional) feature vectors.

6.4 Computational tradeoff: m^3 vs. d^3

This is the practical headline. Linear PCA eigendecomposes a $d \times d$ matrix: cost $O(d^3)$, independent of the number of samples m . Kernel PCA eigendecomposes the $m \times m$ Gram matrix: cost $O(m^3)$, independent of the **dimension** d (and of the feature dimension, which may be infinite).

- Few samples, huge dimension ($m \ll d$, or infinite-dimensional features): kernel PCA wins — and is the **only** option for infinite feature spaces.
- Many samples, modest dimension ($m \gg d$): linear PCA wins; the $m \times m$ Gram matrix becomes the bottleneck (storage $O(m^2)$, decomposition $O(m^3)$).

Kernel PCA buys **nonlinearity** — it can straighten curved structure that linear PCA cannot — at the cost of scaling with the dataset size rather than the dimension.

7 Attention as a kernel operation

The Transformer’s attention mechanism can look like an arbitrary engineering recipe. Seen through this course’s lens, it is something familiar: a **kernel smoother** in a learned embedding space. You already hold every piece — inner products as similarity (kernels), the softmax and its temperature (MAT41-3), residual connections and layer normalization (MAT41-3).

7.1 Scaled dot-product attention

Each input token is mapped, by learned linear projections, to three vectors: a **query** $\mathbf{q}$, a **key** $\mathbf{k}$, and a **value** $\mathbf{v}$, all in $\mathbb{R}^{d_k}$ (values may have their own dimension). Attention computes, for a

query, a **weighted average of values**, where each value’s weight measures how similar its key is to the query.

Definition 14 (Scaled dot-product attention) For a query $\mathbf{q}$ and stacked keys/values $\mathbf{K}, \mathbf{V}$ (one row per token),

$$\text{attn}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_j w_j \mathbf{v}_j, \quad \mathbf{w} = \text{softmax} \left(\frac{\mathbf{q}^T \mathbf{k}_j}{\sqrt{d_k}} \right)_j.$$

In matrix form for all queries at once:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}.$$

7.2 The kernel reading

The similarity score $\mathbf{q}^T \mathbf{k}_j$ is precisely a **linear kernel** $\langle \mathbf{q}, \mathbf{k}_j \rangle$ — an inner product in the learned embedding space. Attention is therefore a **kernel-weighted average**: it predicts a query’s output by blending the values of tokens whose keys are kernel-similar to the query, exactly as a kernel smoother predicts y at a new point by averaging nearby training labels. The network **learns** the embedding in which this inner product is a meaningful similarity — the same spirit as choosing a good kernel, but with the feature map trained rather than fixed.

7.3 Softmax as a Boltzmann distribution over keys

The softmax turns raw similarities into a probability distribution over keys:

$$w_j = \frac{\exp(s_j/\tau)}{\sum_{j'} \exp(s_{j'}/\tau)}, \quad s_j = \mathbf{q}^T \mathbf{k}_j.$$

This is exactly the **Boltzmann (Gibbs) distribution** of statistical physics, with the similarity playing the role of negative energy and the scale $\sqrt{d_k}$ playing the role of **temperature** τ (the temperature parameter you met in MAT41-3’s softmax). High temperature (small scores) $\rightarrow$ near-uniform weights, attention spread over many tokens; low temperature (large scores) $\rightarrow$ weights spike on the single most similar key, attention nearly “hard”.

Remark Why divide by $\sqrt{d_k}$? If query and key components are roughly independent with unit variance, the dot product $\mathbf{q}^T \mathbf{k}$ has variance $\approx d_k$, so its magnitude grows like $\sqrt{d_k}$. Without rescaling, large d_k pushes the softmax into its saturated regime where one weight is ≈ 1 and the rest ≈ 0 ; there the gradient of the softmax (MAT41-3) is nearly zero and learning stalls. Dividing by $\sqrt{d_k}$ holds the score variance near 1 regardless of dimension — it is **temperature control**.

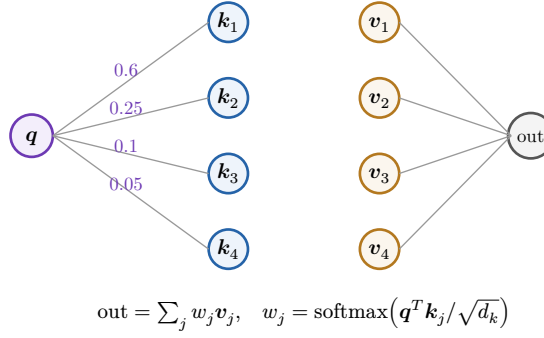


Figure 5: Attention as a kernel smoother. The query is compared to each key by an inner product (a linear kernel); the softmax turns these similarities into weights w_j (a Boltzmann distribution over keys); the output is the weighted average of the values.

7.4 Multi-head attention

A single attention layer applies one learned projection, hence one notion of similarity. **Multi-head** attention runs h attentions in parallel, each with its own learned $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ projections — i.e. h **different learned kernels** — then concatenates the outputs and projects again:

$$\text{MultiHead} = [\text{head}_1, \dots, \text{head}_h] \mathbf{W}^O, \quad \text{head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V).$$

Each head can specialize in a different relationship (one for syntax, one for coreference, ...), much as an ensemble of kernels captures different structure.

7.5 The Transformer block as a mathematical object

The full block composes attention with the pieces you already know from MAT41-3 (here reviewed, not introduced). With input $\mathbf{x}$ (a sequence of token vectors):

$$\begin{aligned} \mathbf{x}' &= \mathbf{x} + \text{MultiHead}(\text{LN}(\mathbf{x})), \\ \mathbf{y} &= \mathbf{x}' + \text{FFN}(\text{LN}(\mathbf{x}')), \end{aligned}$$

where $\text{FFN}(\mathbf{u}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{u} + \mathbf{b}_1) + \mathbf{b}_2$ is a position-wise two-layer network.

Each sub-layer is wrapped in two devices from MAT41-3:

- **Residual connection** $\mathbf{x} + f(\mathbf{x})$: the identity-plus-perturbation structure that lets gradients flow through deep stacks without vanishing.
- **Layer normalization** LN: standardizes each token vector to stabilize the scale of activations entering each sub-layer.

So a Transformer block is the **mathematical composition**

$$\mathbf{y} = (\mathbf{I} + \text{FFN} \circ \text{LN}) \circ (\mathbf{I} + \text{MultiHead} \circ \text{LN})(\mathbf{x}),$$

a kernel-based mixing step (attention) followed by a per-token nonlinear map (FFN), each protected by a residual path and a normalization. Stacking many such blocks builds the Transformer.

8 Reinforcement learning

In supervised learning a teacher provides the right answer for each input. In **reinforcement learning** (RL) there is no teacher — only a **reward** signal earned by acting in an environment over time. The agent must learn **which actions lead to high long-term reward**. In MAT31-2 you met the Markov property and wrote the Bellman equation as a one-step accounting identity on a single small example. Here that identity becomes a family of **algorithms**.

8.1 The Markov decision process

Definition 15 (Markov decision process (MDP)) An MDP is $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: a set of **states** $\mathcal{S}$, **actions** $\mathcal{A}$, a transition kernel $P(s' | s, a)$ (the Markov property: the next state depends only on the current state and action), a reward $R(s, a)$, and a **discount factor** $\gamma \in [0, 1)$.

A **policy** $\pi(a | s)$ specifies how the agent acts. We measure a policy by the **value function** — the expected discounted return from a state:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right].$$

The discount γ makes the infinite sum converge and encodes “a reward now is worth more than the same reward later”. The companion **action-value** $Q^\pi(s, a)$ is the expected return from taking a in s and then following π .

8.2 The Bellman equations, now as fixed points

The one-step identity of MAT31-2 generalizes to two equations. The **expectation** equation evaluates a fixed policy; the **optimality** equation characterizes the best policy.

Definition 16 (Bellman optimality equations)

$$\begin{aligned} V^*(s) &= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right], \\ Q^*(s, a) &= R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a'). \end{aligned}$$

Read $V^*(s)$ as “value today = best (immediate reward + discounted value of where I land tomorrow)” — the exact slogan from MAT31-2, now with the maximization made explicit. The optimal policy is **greedy** with respect to Q^* : $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$.

8.3 Solving a known MDP: value and policy iteration

When P and R are **known**, two algorithms compute V^* by turning the Bellman equation into an update rule.

Definition 17 (Value iteration) Initialize V_0 arbitrarily; repeat the **Bellman backup** until convergence:

$$V_{k+1}(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_k(s') \right].$$

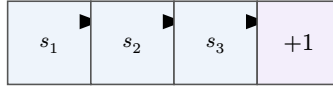
Definition 18 (Policy iteration) Alternate two steps until the policy stops changing:

1. **Policy evaluation:** compute V^π for the current policy π (solve the linear Bellman expectation equations).
2. **Policy improvement:** set $\pi(s) \leftarrow \operatorname{argmax}_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s')]$.

Convergence intuition. The Bellman optimality operator is a **contraction** with factor γ : each backup shrinks the distance to V^* by at least γ ,

$$\|V_{k+1} - V^*\|_\infty \leq \gamma \|V_k - V^*\|_\infty.$$

Since $\gamma < 1$, the error decays geometrically and value iteration converges to the unique fixed point V^* from any start. (The contraction-mapping **proof** is deferred to the Advanced track; the **intuition** — repeated discounting drags errors to zero — is what you need here.) Policy iteration converges in **finitely many** steps because there are finitely many policies and each improvement step strictly improves the value until optimality.



move right $\rightarrow$ reward +1 at the goal; γ discounts distance

Figure 6: A 1×4 grid-world MDP. From any cell the agent may move left or right; reaching the goal pays +1. Value iteration propagates the goal's value leftward: $V^*(s_i) = \gamma^{\text{dist to goal}}$, each backup pushing value one cell further left.

8.4 Learning without a model: bootstrapping and Q-learning

Value and policy iteration need the model (P, R) . Often we do not have it — we can only **act** and observe transitions (s, a, r, s') . **Model-free control** learns directly from this experience.

The key device is **bootstrapping**: update an estimate using **other current estimates** rather than waiting for a final outcome. Q-learning bootstraps the Bellman optimality equation, replacing the unknown expectation by a single observed sample and nudging the estimate toward it:

Definition 19 (Q-learning update) On observing a transition (s, a, r, s') , update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{TD target}} - Q(s, a) \right],$$

where $\alpha \in (0, 1]$ is the learning rate.

The bracket is the **temporal-difference (TD) error**: the gap between the current estimate $Q(s, a)$ and a one-step-better **target** built from the observed reward plus the discounted best next value. We move $Q(s, a)$ a fraction α of the way toward the target — gradient-descent-like nudging (MAT61-1) on a moving target.

Worked example — one Q-learning update. Suppose $Q(s, a) = 2.0$, and from (s, a) we observe reward $r = 1$ and land in s' where the current estimates are $Q(s', \text{left}) = 3.0$ and $Q(s', \text{right}) = 5.0$. With $\gamma = 0.9$ and $\alpha = 0.1$:

$$\text{target} = r + \gamma \max_{a'} Q(s', a') = 1 + 0.9 \cdot 5.0 = 5.5,$$

$$Q(s, a) \leftarrow 2.0 + 0.1(5.5 - 2.0) = 2.0 + 0.35 = 2.35.$$

The estimate rose toward the more optimistic target.

8.5 Exploration and off-policy learning

If the agent always takes the action it currently believes is best, it may never discover a better one. ϵ -greedy exploration resolves this: with probability $1 - \epsilon$ take the greedy action $\operatorname{argmax}_a Q(s, a)$ (**exploit**); with probability ϵ take a uniformly random action (**explore**). Typically ϵ is annealed toward 0 as learning progresses — explore early, exploit once confident.

Definition 20 (On-policy vs. off-policy) An **off-policy** method learns the value of one policy (the **target** policy) while behaving according to another (the **behaviour** policy).

Q-learning is **off-policy**: the behaviour policy is ϵ -greedy (it sometimes explores randomly), but the update target uses $\max_{a'} Q(s', a')$ — the value of the **greedy** target policy, not of the action actually taken next. This decoupling is what lets Q-learning explore freely while still converging to the **optimal** greedy policy Q^* .

Common pitfall Do not confuse the **behaviour** and **target** policies. The max in the Q-learning target is what makes it off-policy and optimal; replacing it with the value of the actually-taken next action gives a **different**, on-policy algorithm (SARSA) that evaluates the exploratory policy instead. The ϵ belongs to the behaviour policy (how we act); the max belongs to the target (what we learn).

The arc of this chapter. Known model $\rightarrow$ solve the Bellman equation directly by value/policy iteration (contraction guarantees convergence). Unknown model $\rightarrow$ bootstrap from sampled transitions with Q-learning, exploring via ϵ -greedy and learning off-policy. In every case the engine is the same Bellman accounting identity you first wrote in MAT31-2.

9 Exercises

Exercises

1. **(Bias-variance.)** You fit polynomials of increasing degree and observe that training error keeps falling while test error falls then rises. Identify which of bias, variance, and irreducible error each trend reflects, and explain why no model can drive test error to zero on noisy data.
2. **(Ridge.)** Using the SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, show that the ridge filter factor on singular direction i is $\sigma_i^2/(\sigma_i^2 + \lambda)$. What happens to a direction with $\sigma_i = 0$, and why does this cure the ill-conditioning of ordinary least squares?
3. **(L1 vs L2.)** Explain geometrically why the lasso produces exact zeros while ridge does not. In a problem with 1000 features of which you believe only 10 matter, which penalty would you choose, and why?
4. **(Cross-validation.)** A colleague standardizes the full dataset, then runs 5-fold CV to pick λ , then reports the CV error as the final performance estimate. Name the two methodological errors and correct the procedure.
5. **(Kernels.)** Decide whether each is a valid kernel and justify via the Gram matrix or the construction rules: (a) $k(x, z) = (2 + x^T z)^3$; (b) $k(x, z) = \exp(-\gamma\|x - z\|^2)$; (c) $k(x, z) = x^T z - 5$.
6. **(SVM.)** Write the hard-margin primal and its dual. Point to the exact KKT condition that makes most $\alpha_i = 0$, and explain why the dual — not the primal — is the natural place to apply the kernel trick.
7. **(Soft margin.)** Describe the effect of $C \rightarrow \infty$ and $C \rightarrow 0$ on the margin width and on the bias-variance tradeoff. Why must C and the RBF γ be tuned jointly?
8. **(Kernel PCA.)** State the cost of linear PCA and of kernel PCA in terms of d and m , and give one regime where each is preferable. Why must we center the Gram matrix rather than the feature vectors?
9. **(Attention.)** Explain attention as a kernel operation, identifying the kernel, the role of the softmax, and the reason for the $1/\sqrt{d_k}$ scaling. What goes wrong if the scaling is omitted for large d_k ?
10. **(RL.)** For the example with $Q(s, a) = 4.0$, observed $r = -1$, next-state estimates $Q(s', \cdot) \in \{2.0, 6.0\}$, $\gamma = 0.9$, $\alpha = 0.5$, compute one Q-learning update. Then explain why Q-learning is off-policy and how ε -greedy balances exploration and exploitation.